

Reimagining Probability Storage and Variable Assignments in DuBio: A Comparative Study of Data Structures

THOMAS MAAS, University of Twente, The Netherlands

Merging and inserting data from multiple sources can cause problems such as duplication, multiple labels referring to the same source and other data quality issues. Probabilistic databases view such "uncertain" data as a valuable aspect instead of a problem, and use intrinsic probabilities to draw conclusions. This paper discusses, provides and analyzes solutions to a bottleneck regarding dictionary lookups in PostgreSQL extension DuBio for probabilistic data. A black box explanation of DuBio is provided, and the quick growth of variables in their dictionary structure is addressed. Suitable solutions within the scope of this research are a table of tuples or a table of triplets for storing variables instead of a dictionary, which can be used to calculate conjunctions of propositional formulas called sentences. An algorithm for those calculations using only PostgreSQL is provided as a high level diagram, pseudocode and full queries are provided in the appendix. The performance analysis reveals the newfound methods to be almost strictly better, only being outperformed by DuBio's dictionary lookups combined with binary decision diagrams, in very small datasets.

Additional Key Words and Phrases: Probabilistic Databases, Probability Computation In PostgreSQL, PostgreSQL, DuBio, Synthetic Data

1 INTRODUCTION

The growing complexity of real-world data has underscored the limitations of traditional deterministic databases. Dealing with real-world data often brings about data quality issues such as missing values, duplicates or conflicting information, which traditional precise databases are not equipped to deal with. These issues arise during e.g. the integration of multiple sources of data, imperfect extraction of data or missing data.

A probabilistic database is a Database Management System (DBMS) capable of storing, querying and manipulating uncertain data [4]. These databases explicitly represent uncertainty in their data, and consider it an important property of the database [5]. These databases are a probability space over the possible contents of the database. Database entries can be seen as a tuple, and as either existing or non existing in a realized version of the database, a single collapsed "world" [4]. We call this the possible worlds theory, where we model properties of data, such as its value or label, as existing in a possible world [2]. DuBio is an extension of PostgreSQL currently under development at the University of Twente in Enschede. It implements a probabilistic layer on top of PostgreSQL using Binary Decision Diagrams and allows the user to manipulate and query uncertain data [9]. Currently, DuBio stores random variable values and their respective probabilities (e.g. $X=1: 0.8$, $X=2: 0.2$) in a dictionary contained in a single database row, this dictionary contains a list of all variables

relevant to a given dataset[9]. An example of the current inner workings of DuBio regarding usage of dictionary entries can be found in Section 2. The author of DuBio has indicated that this method of storage is expected to become a bottleneck in system performance.

This research addresses the concern of the expected bottleneck by creating two variants of an alternative method for storing and calculating probabilities, and performing a comparative analysis of execution time under varying circumstances. Circumstances of interest are e.g. the amount of rows, columns or variables used in computations. The implemented variants show major improvements in execution time for larger databases, despite being a PostgreSQL only solution compared to DuBio's solution implemented in C. Besides the implementation and analysis of aforementioned methods, this research also discusses possible explanations for unexpected execution time changes in later sections.

2 BACKGROUND

In order to better understand DuBio's inner workings and why the dictionary structure poses a problem, we consider the following example. A car sales company went through data integration of multiple locations' datasets. These locations have labeled their data differently, although they refer to the same car brand. This produces the following table.

	Car brand	Sales	
4	Mercedes	67	$X=0$
5	Mercedes-Benz	39	$X=0$
	Mercedes	106	$X=1 \wedge Y=0$
	Mercedes-Benz	106	$X=1 \wedge Y=1$
6	Renault	45	$\top$

Fig. 1. Example database entries for a car company

We notice that both locations have labeled their sales of the car brand Mercedes differently, this is clear in this example, but might not be as obvious more complex real-world situations. As mentioned in Section 1, we model the possibilities and uncertainty in the data using the possible worlds theory. This means we have world $X=0$ and $X=1$, where the car brands are different or the same in reality, respectively. If the car brands are the same, so $X=1$, the correct name for the merged database entry is also uncertain. This gives us another two worlds, $Y=0$ and $Y=1$. These worlds are often not equally likely, so their respective probabilities are stored in the before-mentioned dictionary. An example of such an entry can be found below.

Looking at Figure 1, we consider the third row and its assigned variables. The unresolved collection of variables and logical operators

Author's address: Thomas Maas, t.p.maas@student.utwente.nl, University of Twente, P.O. Box 217, Enschede, The Netherlands, 7500AE.

TScIT 43, July 4, 2025, Enschede, The Netherlands

© 2025 ACM.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Proceedings of 43th Twente Student Conference on IT (TScIT 43)*, <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>.

name	dict
'mydict'	dictionary('X=0:0.4; X=1:0.6; Y=0:0.7; Y=1:0.3')

Fig. 2. Example dictionary entry for a car companies' DuBio database.

is called a propositional formula:

$$X = 1 \wedge Y = 0$$

which will be referred to as a *sentence* in this research and DuBio's documentation. Using this sentence and the given dictionary of probabilities, we can calculate the probability of this world being the "real world", where Mercedes and Mercedes-Benz refer to the same brand, with the actual name being Mercedes and combined sales of 106:

$$P(X = 1) * P(Y = 0) = 0.6 * 0.7 = 0.42.$$

It is easily seen that given a sufficiently large database, the length of a sentence and thus the amount of random variables can rapidly grow to be enormous. The resolving of sentences in DuBio is done using a Binary Decision Diagram (BDD) traversal algorithm in C. Such a diagram represents the truth table of sentence. The newly implemented methods in PostgreSQL use strictly different algorithms. As such, only a black box understanding of DuBio is required for both the reader and author of this paper.

3 PROBLEM STATEMENT

As illustrated in Section 2, the amount of possible worlds depends on the dataset. The calculation of a single probability becomes increasingly difficult with larger probabilistic data, as the amount of possible worlds increases, causing combinatorial increase in complexity. The current implementation of DuBio stores variables (possible worlds) in a dictionary within a single database row, which is expected to become a performance bottleneck during probability calculation, which lies at the core of probabilistic query evaluation. Since probabilities are computed per tuple, repeated dictionary lookups (using hash tables) in C introduce significant overhead due to poor cache locality and lack of planner-level optimization [3]. An alternative solution using PostgreSQL allows the usage of joining tables, possibly increasing performance of probability calculation with large volumes of data.

To address this, a comparative analysis is conducted between DuBio's current approach and two variants of an alternative method for storing and computing probabilities. This research focuses on execution time under varying conditions, such as dataset size, query complexity, and the number of variables involved. The study aims to answer the following research questions regarding the design, implementation, and performance trade-offs of these methods.

3.1 Main research question

How do alternative implementations of probability storing data structures compare to DuBio's current dictionary in terms of performance during probability calculation across varying query and data complexity scenarios?

3.2 Research question 1

What are alternative data structures that could more efficiently store probabilities in a probabilistic database system?

3.3 Research Question 2

How can these alternative structures be implemented using only PostgreSQL queries, and what are the limitations of such implementations?

3.4 Research Question 3

Which dataset properties significantly affect the execution time of queries, and how do they influence the performance of each method?

4 RESEARCH SCOPE

As mentioned in Section 2, DuBio resolves sentences using BDD's. This type of directed acyclic graph (DAG) allows for use of any logical operator, and as such any type of sentence. Considering the difficulty of resolving such sentences in a query based language like PostgreSQL, we decide to focus on conjunctions. A conjunction is a sentence using only AND operators, resulting in a sentence such as $X = 0 \wedge Y = 1 \wedge Z = 2 \wedge \dots$ which resolves to a formula like $0.3 * 0.2 * 0.6 * \dots$. While this research will work with conjunctions only, the performance a more general probability calculation function is not expected to be much different from computing the probability of a conjunction. A possible workaround using DuBio's dictionary and description thereof is provided in Section 10.

5 METHODOLOGY AND CONTRIBUTIONS

5.1 Choice of data structures

Suitable alternative data structures are limited to solutions developed in PostgreSQL as the research scope, timeframe and relevant experience do not allow development of a new extension mimicking DuBio. Given this constraint, the amount of feasible solutions becomes limited. Such solutions can be but are not limited to:

- Variable storage in a table of tuples, e.g., ('A=1', 0.5)
- Variable storage in a table of triplets, e.g., ('A', 1, 0.5)
- Variable storage in JSONB (native PostgreSQL type), e.g., {"A=1": 0.5}
- Transformation of relevant variables for a calculation to a new temporary DuBio dictionary instead of performing lookups in a global dictionary

Considering the need to analyze execution time for alternative methods, the temporary dictionary is an interesting yet not fully suitable method for this research' goals. Details of such an implementation and expected results will be discussed in Section 10. JSONB offers functionality and future proof flexibility such as efficiently searching key value pairs, array style subscripting expressions and built in mapping to dictionaries, lists and scalars in the widely used programming language Python [7]. For this research, a more bare bone implementation decreases overhead and provides a more isolated style experiment. A single-purpose isolated experiment is most appropriate when considering the measurement of execution time. This leads to the decision of implementing variable storage and calculation of probabilities using only tables of tuples and triplets in

PostgreSQL. Given variables A and B, with values 1 and 2 for both, those tables would look like:

VarVal	Prob
A=0	0.3
A=1	0.7
B=0	0.6
B=1	0.4

Table 1. Tuple Variable Table

Note: VarVal denotes the variable name combined with its value, separated by a '=' and saved as a string.

Var	Val	Prob
A	0	0.3
A	1	0.7
B	0	0.6
B	1	0.4

Table 2. Triplet Variable Table

Note: Var denotes the variable name saved as a string, and Val denotes its corresponding value saved as an integer.

5.2 A query-based algorithm for resolving conjunctions

As seen in Section 4, a conjunction is a chain of variables which can be resolved by multiplying all values in the chain. As all sentences in the datasets are conjunctions, we need to extract variables from a sentence and multiply their probabilities, after which they are joined back on the original dataset. This can be achieved by manipulating tables of the dataset using a single big PostgreSQL query. A high level visual overview of this process using a tuple table like Table 1 is presented in Figure 3. A full implementation of the algorithm in query form can be found in appendix A.

5.3 Pseudocode

Pseudocode describing the algorithm using either a tuple or triplet table for variable storage is provided in Algorithm 1.

6 EXPERIMENTAL SETUP

In order to perform experiments measuring server side execution time under varying conditions, a flexible tool that creates datasets based on relevant parameters is necessary. Such parameters are:

- Amount of rows
- Amount of columns
- Amount of variables per dataset

After creating a dataset with given parameters, the tool should also:

- Create a DuBio dictionary based on the dataset
- Create a table of tuples based on the dataset
- Create a table of triplets based on the dataset
- Create a valid sentence for each row based on relevant variables.

Algorithm 1 Compute Sentence Probabilities Using Tuple or Triplet Variable Tables

```

1: procedure COMPUTEPROBABILITIES(table_name)
2:   Extract unique sentences from table_name, give them an ID
3:   for all sentences do
4:     Extract VarVal pairs in sentence (e.g., (A=1), (B=1) for
       tuple variables and (A,1),(B,1) for triplet variables ) using
       regular expressions
5:     if sentence is constant 1 then
6:       Assign probability 1.0
7:     else
8:       Add VarVal pairs to table containing all unique
       pairs
9:       Add (sentence_id, VarVal) to a results table
10:    end if
11:  end for
12:  join the tuple or triplet variables table join the table with
    unique pairs
13:  join created table with unique (VarVal, prob) or
    (var, val, prob) ON the results table. Resulting in
    (sentence_id, prob)
14:  for all sentence_id do
15:    Multiply all probabilities with sentence_id using
    EXP(SUM(LN(prob)))
16:  end for
17:  for all sentence_id do
18:    Replace sentence_id with sentence
19:  end for
20:  Return join of table_name and (sentence, probability) on
    sentence = sentence
21: end procedure

```

As is seen in Figure 1 in Section 2, a variable is a property and the value is a specific realization of that property. In the example, X denotes "Mercedes and Mercedes-Benz are the same" and the value denotes whether that is true (1) or false (0). Whereas Y denotes the true name being "Mercedes" if Y=0 or "Mercedes-Benz" if Y=1. The property X and Y are only relevant to the first four entries, as they are not a property of "Renault". Using this logic and using names instead of car brands, we can generate a dataset based on people and their physical traits such as hair color and length follows:

- (1) Choose the amount of people in your dataset
- (2) Choose properties $x = [\text{hair color, length}]$
- (3) Choose values for each property $y = [[\text{brown, gray, black, blonde}], [\text{short, average, tall}]]$

Adding Maria and Jack with 2 and 3 entries respectively generates the following example dataset:

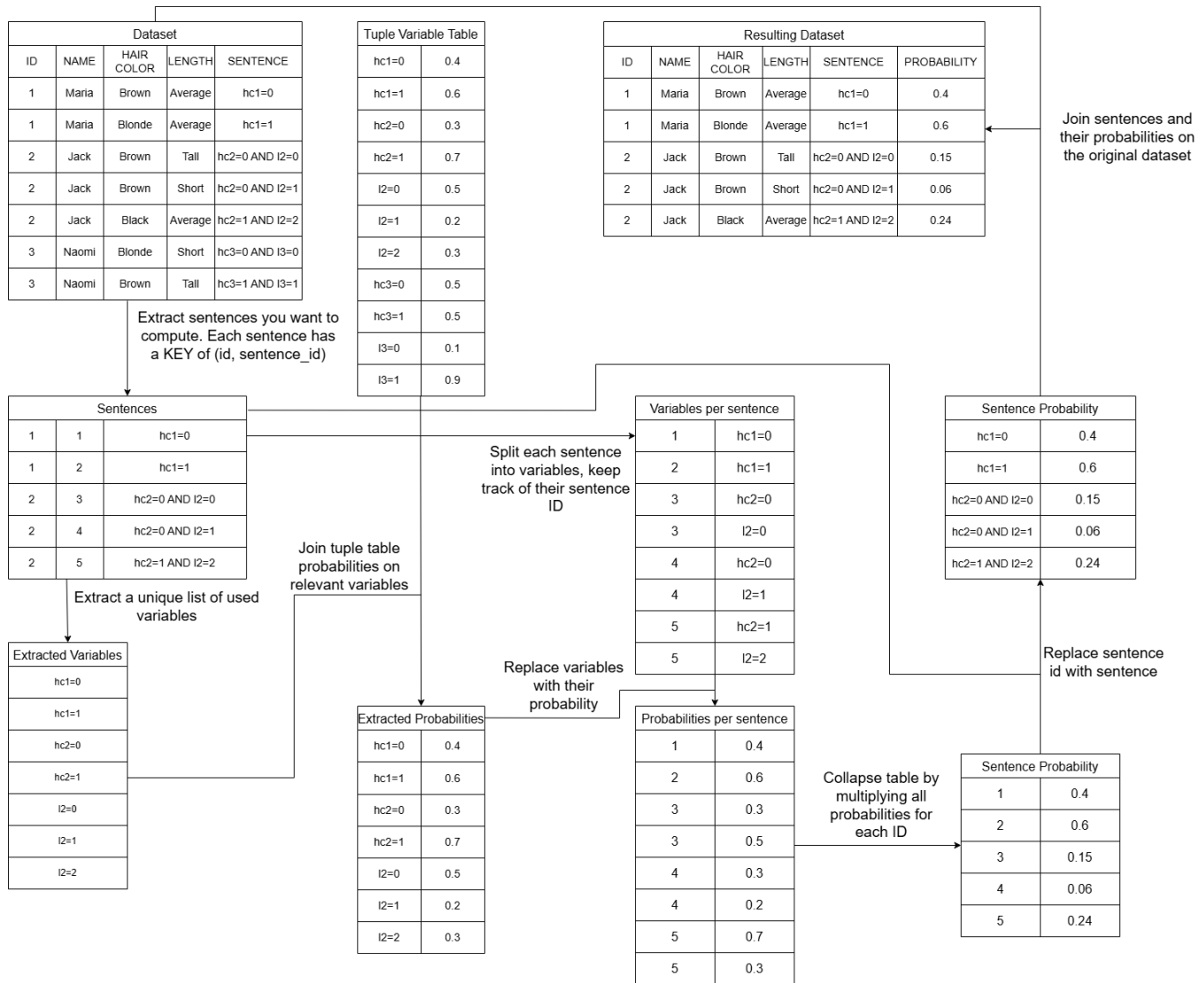


Fig. 3. Computing probabilities given a dataset and a table of tuples containing variables and probabilities.

ID	name	hair color	length	sentence
1	Maria	Brown	Average	hc1=0
1	Maria	Blonde	Average	hc1=1
2	Jack	Brown	Tall	hc2=0 AND l2=0
2	Jack	Brown	Short	hc2=0 AND l2=1
2	Jack	Black	Average	hc2=1 AND l2=2

Table 3. Example of a generated dataset

Note: variables are generated as "id" + "property" = value. It is important to observe that Maria has no variable "l1" as her length is the same in both entries, meaning her length is certainly "average".

During the generation of a dataset, occurrences of variables and their values are stored in a DuBio dictionary and a tuple table similar to Table 1 and a triplet table similar to Table 2. The variables are assigned random probabilities such that for property x with values $0, \dots, n$:

$$X = 0' + \dots + X = n' = 1$$

An example of actual generated data using abstract names and values can be found in appendix B.

7 ANALYSIS TERMINOLOGY AND MEASUREMENTS OF INTEREST

Execution time: A server side measurement of execution time is necessary to prevent any overhead regarding connection times and formation of queries. PostgreSQL provides the *EXPLAIN* command. Along with option *ANALYZE* it performs the query and shows run-time alongside other statistics [6].

Rows returned: A server side measurement of expected returned amount of rows also by *EXPLAIN ANALYZE*. Not to be confused with rows in a dataset. This value is used to assess fairness of query comparison, as they should all return the same amount of rows (sanity check).

Variables in dictionary: A dictionary has an amount of variables and a number of values for each variable. The example in Table 3 shows 3 visible variables and 7 values. The range of values in the example dataset is [4, 10]. Internally and not shown, a 4th variable $l1=0$ exists with a probability of 1. The amount of variables in generated synthetic data can be calculated as follows: $unique_ids * (columns - 3)$. The amount of generated variables are deterministic in contrast to values.

Amount of rows (or sentences): The amount of rows in a table passed to a query. This can differ from the total rows in a dataset as the input must be a subset of the entire dataset. An increase in rows affects the amount of variables and values to a degree depending on the amount of unique ID's which we will call ID density. This term is explained in detail later in this section.

Amount of columns: Generated synthetic data always has a minimum of 4 columns: (*ID*, *name*, *variable*, *sentence*). Additional columns always increase the amount of variables and values. Additional columns extend a database horizontally compared to vertical increase with rows.

ID density: ID Density is often referred to in this paper. It gives an estimate of the expected ratio between dictionary variables and values. In Table 3 we see 4 variables ("l1" is hidden as it is a certainty) and 8 values. It is important to note that the generated data discussed in Section 6 can not have a different amount of rows per ID as seen in Table 3. This is to maintain a deterministic nature of experiments, where no randomness is involved regarding distribution of rows across ID's. Every ID in a generated table has a variable in the dictionary for each of their properties (columns). When an ID has multiple rows, the amount of values increases in contrast to the constant amount of variables. In a table of 1 ID, 5 rows and 5 columns, 5 variables and a maximum of 25 values exist. In a table of 5 ID's, 5 rows and 5 columns, 25 variables exist. Such a situation means every ID only has 1 row, meaning every row is already certain, this is referred to as the trivial case. Thus, an increase in unique ID's increases the amount of variables in an experiment. More unique ID's increase ID density.

8 RESULTS

8.1 Observations

Amount of rows vs. Execution time for varying amounts of variables in dictionary: It is observed that execution time scales approximately linearly with increasing rows of data as shown in Figure 4. Each figure shows the same experiment with a different amount of variables in the dictionary. When increasing this value, the execution times grow faster yet linearly. It is clear that an increasing amount of variables on a small scale already affects the degree of scaling for DuBio significantly, while tuple and triplets are almost unaffected at this datasize. This prompts following experiments to focus on the increase in dictionary variables and their effect on execution time instead of an increase in the amount of rows.

Variables in dictionary increasing through ID density vs. Execution time: Figure 5 shows the results of increase in variables by means of adding more unique IDs and maintaining the same amount of rows. It produces sporadic results for DuBio's execution time while tuple and triplets maintain a stable execution time. Possible reasons for this phenomenon are discussed in Section 8.2. The rightmost sample points are of 2000 variables. This is a small amount of variables compared to the experiment shown in Figure 7 and real-world data [1]. This experiment shows that even on small scale experiments, the amount of variables are a major, if not the most significant, influence on a query's execution time.

Variables in dictionary increasing through amount of columns vs Execution time: Figure 6 shows a different type of scaling compared to Figure 5. The difference between the figures is the increase of variable amount by adding columns vs. changes in ID density, respectively. While still stable, a greater increase in execution time for tuple and triplet storage is observed in comparison to 5. A possible reason for this is an increase in complexity regarding PostgreSQL JOIN usage, further discussed in Section 8.2. Furthermore, triplet storage does not outperform DuBio compared to Figure 5 despite having the same amount of variables across sample points. This indicates a possible worst case complexity scenario for tuple and triplet methods in wide databases (many columns compared to rows).

Large amount of variables in dictionary increasing through ID density vs. Execution time on a logarithmic axis: Figure 7 reveals DuBio's incapability of managing large volumes of data on a logarithmic scale. At a breaking point somewhere between 100 and 1000 variables, execution time increases significantly. It is probable that the same breaking point is observed in Figure 5 around 300 variables. This occurrence is discussed in Section 8.2. This scenario has been tested with up to 60000 variables due to internal limits of DuBio, of which an explanation is given in Section 8.2. In a real-world scenario, many more variables could be present [1], underscoring the severity and reality of the formerly theorized bottleneck. Tuple and triplet calculation show no significant increase in execution time at any point, maintaining the same stable performance as observed in Figure 5. An interesting observation is decrease in execution time when nearing the maximum amount of variables. As mentioned in Section 7, a trivial case occurs when the amount of rows equals the amount of unique ID's, having all sentences simply be true. In this case, the highest amount of possible

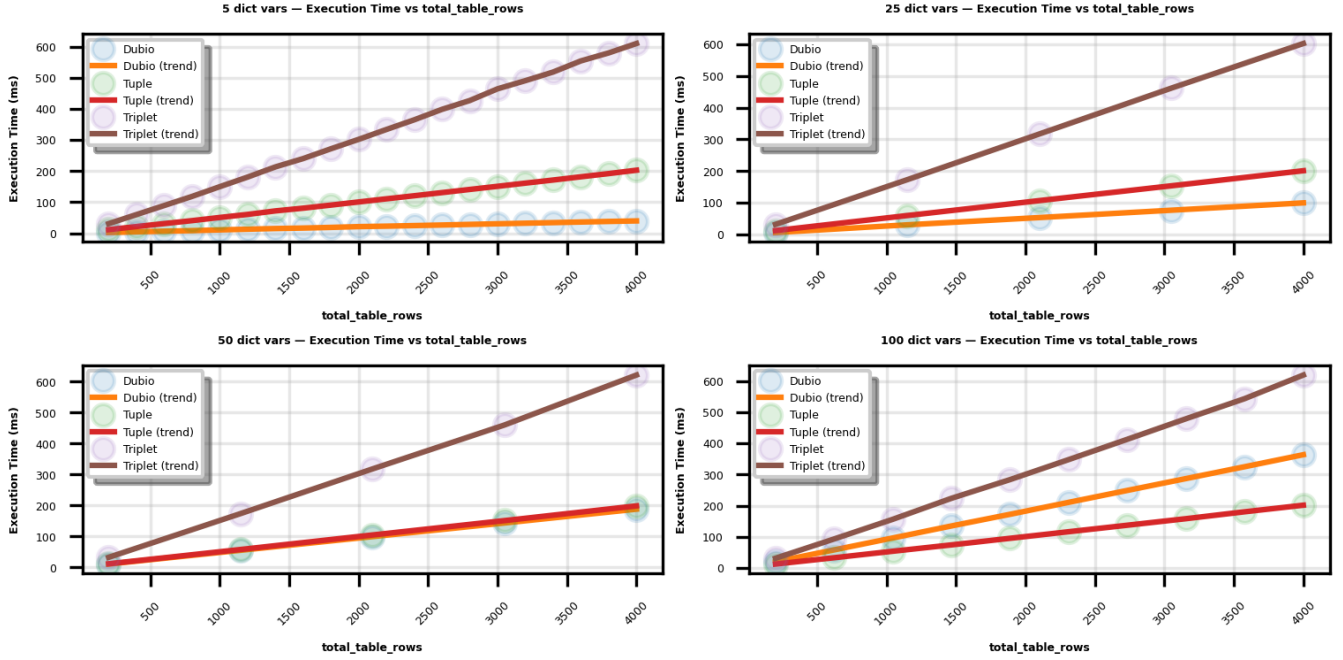


Fig. 4. Amount of rows vs. Execution time for varying amount of variables in dictionary

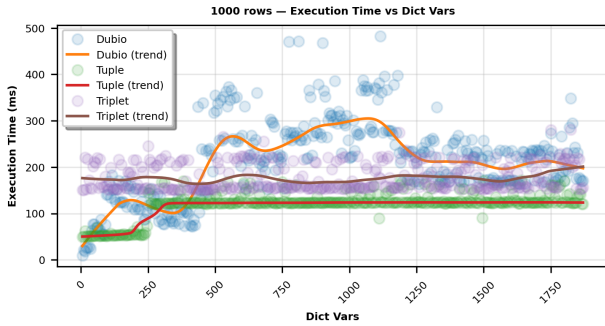


Fig. 5. Initial changes in execution time with an increasing amount of variables through ID density, starting from 0

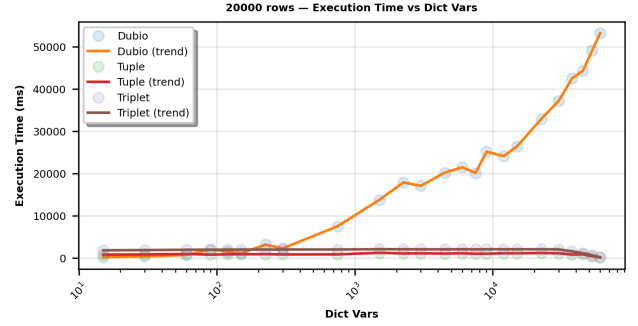


Fig. 7. Large amount of variables increasing through ID density vs Execution time on a logarithmic axis

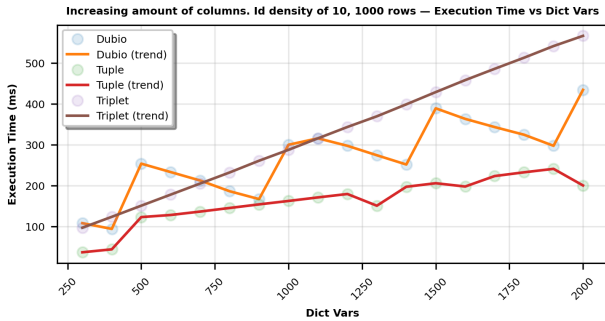


Fig. 6. Variables in dictionary increasing through column amount vs. Execution time

variables for a given database shape (rows, columns) is achieved. DuBio shows no improvement, while tuple and triplet execution time decreases, albeit a relatively small difference.

8.2 Discussion

The resulting graphs provide insight in query performance across varying data complexity scenarios, while also revealing limitations of DuBio.

Tuple vs Triplet algorithm: The tuple storage of variables strictly outperforms triplet storage. This is likely because triplet storage requires an additional string split of keys to convert from tuple to triplet e.g. ('A=1', 0.5) to (A, 1, 0.5). Furthermore, an additional JOIN is required for the extra column in (var, val, prob) compared to (var=val, prob) as seen in appendix A or derived from Figure 3.

It was hypothesized that a string comparison with keys like 'A=1' may be performance costly in case of many comparisons, compared to a smaller string 'a' and a number '1'. Clearly, the smaller string comparison does not outweigh downsides of the triplet algorithm. Those downsides are the aforementioned extra JOIN, additional string split and possibly having a composite key (a, 1) compared to the single primary key ('A=1') in tuple storage.

Limitations of a DuBio Dictionary: During generation of the large dataset in Figure 7, the largest possible x value, or variables in the dictionary, was 65535. This corresponds to the limit of an unsigned 16-bit integer, meaning a DuBio dictionary can not hold any more variables than that. This would prevent deployment of DuBio on large-scale datasets. An example is a questionnaire by the US census bureau having over 50 columns and approx. 12.5 million tuples (rows) in their dataset [1]. This limit does not necessarily mean anything performance-wise, yet is a good example of the need for a revised probability calculation algorithm as presented in this paper.

Limitations of the Tuple and Triplet algorithm: As tuple and triplet variable storage utilize standard PostgreSQL tables, the storage is limited by PostgreSQL standards. The maximum row size is 4,294,950,911 - an unsigned 32 bit integer [8]. The used example of large real-world data presented in [1] indicates approximately 12 million tuples or rows. It can be concluded that the current maximum amount of rows is more than adequate to have the tuple and triplet act on probabilistic data. It is important to note that if the need arises, this limit may be circumvented in future work by adapting the query to be able to use multiple variable storage tables and splitting tasks in smaller segments of the maximum number of rows.

Explaining the erratic differences in DuBio execution time in smaller datasets: Figure 5 and 7 shows shifts in DuBio execution time under lower amount of variables. This could be attributed to caching behavior, which attempts to catch up with demands during subsequent lookups. A factual statement about this can not be made as the project scope is limited to a black-box overview of DuBio, yet this type of behavior is known in dictionary lookups using hash tables with growing data [3]. Further work might provide insight into the observed pattern in execution times.

9 CONCLUSION

The newly created methods of variable storage both greatly outperform DuBio when calculating conjunctions in most scenarios. DuBio performs better than both methods with a variable amount lower than 50, an unusual scenario when dealing with real-world probabilistic data. While tuple storage performs better than the triplet storage, their small difference in execution time does not change when increasing data complexity by adding more unique ID's as seen in Figure 7. An increasing difference is observed when the columns are increased in isolation as seen in Figure 6. The effects on execution time when combining these types of data scaling are out of scope for this research. The most significant difference in execution time occurs when structure of the database is changed, e.g. 1 person holding 4 variables and 4 values versus 2 people holding 2 variables and 1 value each. On the contrary, horizontal expansion of

the database seems to not affect the degree of scaling in execution time for any method. Furthermore, a vertical expansion (addition of columns) while maintaining database structure and row amount creates a scenario where DuBio outperforms triplet storage and scales approximately equal to tuple storage as seen in Figure 6. These data complexity scenarios were tested in isolated environments, with changing values for only 1 factor at a time. In a real-world scenario or other complex datasets, one can expect all these factors to change over time. Because the scaling seen in Figure 7 is much greater than the scaling in Figure 6, it can be concluded that tuple and triplet storage will (greatly) outperform DuBio across most query and data complexity scenarios when calculating conjunctions.

Besides the difference in performance, limitations of DuBio such as the relatively small amount of possible dictionary entries discussed in Section 8.2 limit the current use case in real-world scenarios. This highlights possibilities for overall improvement of DuBio by means of development and integration of alternative solutions, such as the tuple and triplet method.

10 FURTHER WORK

Due to time constraints and the scope of the paper, the following points are left for future research.

- A similar comparative analysis of the implementation of functionality to handle all logical operators (OR, NOT, etc..) of propositional statements (sentences) in PostgreSQL.
- A similar comparative analysis of the implementation of smaller temporary DuBio dictionaries based on a relevant variables in a table of sentences, then using DuBio's native functionality to resolve sentences using the smaller dictionary. As the size of the dictionary poses performance issues, such a decrease in size could show promising results, while maintaining all DuBio functionality.
- The optimization of the queries using the tuple and triplet storage.
- Further experimentation with other data complexity scenarios or changes in multiple factors at a time instead of isolated experiments.

REFERENCES

- [1] Lyublena Antova, Christoph Koch, and Dan Olteanu. 2006. 10^{10} Worlds and Beyond: Efficient Representation and Processing of Incomplete Information. *CoRR* abs/cs/0606075 (2006). arXiv:cs/0606075 <http://arxiv.org/abs/cs/0606075>
- [2] Nilesch Dalvi, Christopher Ré, and Dan Suciu. 2009. Probabilistic databases: diamonds in the dirt. *Commun. ACM* 52, 7 (July 2009), 86–94. <https://doi.org/10.1145/1538788.1538810>
- [3] Tim Gubner, Viktor Leis, and Peter Boncz. 2020. Efficient Query Processing with Optimistically Compressed Hash Tables Strings in the USSR. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 301–312. <https://doi.org/10.1109/ICDE48307.2020.00033>
- [4] Maurice Van Keulen. 2018. *Probabilistic Data Integration*. Springer International Publishing, Cham, 1–9. https://doi.org/10.1007/978-3-319-63962-8_18-1
- [5] Matteo Magnani and Danilo Montesi. 2010. A Survey on Uncertainty Management in Data Integration. *J. Data and Information Quality* 2, 1, Article 5 (July 2010), 33 pages. <https://doi.org/10.1145/1805286.1805291>
- [6] PostgreSQL. [n. d.]. EXPLAIN. <https://www.postgresql.org/docs/current/sql-explain.html>. Accessed: 23-06-2025.
- [7] PostgreSQL. [n. d.]. Json types. <https://www.postgresql.org/docs/current/datatype-json.html>. Accessed: 22-06-2025.
- [8] PostgreSQL. [n. d.]. Limits of PostgreSQL. <https://www.postgresql.org/docs/current/limits.html>. Accessed: 23-06-2025.

[9] Maurice van Keulen. [n.d.]. DuBio github page. <https://github.com/utwente-db/DuBio>. Accessed: 03-05-2025.

A QUERIES

These queries were used to extract variables from sentences and resolve the conjunctions. Tuple and triplet storage is expected to exist in your database.

A.1 Query using tuple storage

Listing 1. SQL query to compute sentence probabilities using a tuple storage table for variables, where variable-value pairs are stored as strings.

```
WITH ranked_sentences AS (
  -- Assign a unique ID to each distinct logical
  sentence
  SELECT DISTINCT toString(_sentence) AS sentence_text,
    DENSE_RANK() OVER (ORDER BY toString(_sentence)) AS sentence_id
  FROM sentences_table
),

sentence_tokens AS (
  -- Extract tokens (e.g., "a=1", "b=2") from each
  sentence
  SELECT rs.sentence_id, token
  FROM ranked_sentences rs,
    LATERAL regexp_split_to_table(rs.sentence_text,
      '([()&!|]+') AS token
  WHERE token <> '' AND token <> '1'

  UNION

  -- Handle the tautology sentence "1" by assigning it
  a probability of 1
  SELECT rs.sentence_id, '1'
  FROM ranked_sentences rs
  WHERE rs.sentence_text = '1'
),

relevant_tokens AS (
  -- Determine all unique tokens used across all
  sentences
  SELECT DISTINCT token
  FROM ranked_sentences rs,
    LATERAL regexp_split_to_table(rs.sentence_text,
      '([()&!|]+') AS token
  WHERE token <> '' AND token <> '1'
),

token_probabilities AS (
  -- Join each relevant token with its stored
  probability
  SELECT rt.token, dp.probability
  FROM relevant_tokens rt
  JOIN dual_relation_table dp ON rt.token = dp.token
),

joined_probabilities AS (
  -- Associate each sentence token with its probability
  SELECT st.sentence_id,
    CASE WHEN st.token = '1' THEN 1.0 ELSE tp.
      probability END AS prob
  FROM sentence_tokens st
  LEFT JOIN token_probabilities tp ON st.token = tp.
    token
),
```

```
aggregated_sentence_probs AS (
  -- Compute total sentence probability as product of
  individual token probabilities
  SELECT sentence_id, EXP(SUM(LN(prob))) AS
    sentence_probability
  FROM joined_probabilities
  GROUP BY sentence_id
),

final_sentence_probabilities AS (
  -- Combine the sentence text with its computed
  probability
  SELECT rs.sentence_text, ap.sentence_probability
  FROM ranked_sentences rs
  JOIN aggregated_sentence_probs ap ON rs.sentence_id =
    ap.sentence_id
)

-- Final result: original sentence ID, sentence string,
and computed probability
SELECT s.id, toString(s._sentence) AS sentence_text, fp.
  sentence_probability
FROM sentences_table s
JOIN final_sentence_probabilities fp ON toString(s.
  _sentence) = fp.sentence_text
ORDER BY s.id;
```

A.2 Query using triplet storage

Listing 2. SQL query to compute sentence probabilities based on variable-value probabilities in a triplet format.

```
WITH ranked_sentences AS (
  -- Assign a unique ID to each distinct logical
  sentence
  SELECT DISTINCT toString(_sentence) AS sentence_text,
    DENSE_RANK() OVER (ORDER BY toString(_sentence)) AS sentence_id
  FROM sentences_table
),

sentence_variables AS (
  -- Extract (variable, value) pairs from each sentence
  SELECT rs.sentence_id, match[1] AS var, match[2]::INT
    AS val
  FROM ranked_sentences rs,
    LATERAL regexp_split_to_table(rs.sentence_text,
      '([()&!|]+') AS token,
    LATERAL regexp_matches(token, '^([a-zA-Z0-9]+)
      =([0-9]+)$') AS match
  WHERE token <> '' AND token <> '1'

  UNION

  -- Special handling for tautology sentence "1"
  SELECT rs.sentence_id, '1' AS var, 1 AS val
  FROM ranked_sentences rs
  WHERE rs.sentence_text = '1'
),

relevant_variable_values AS (
  -- Collect all unique (var, val) pairs that appear in
  any sentence
  SELECT DISTINCT match[1] AS var, match[2]::INT AS val
  FROM ranked_sentences rs,
    LATERAL regexp_split_to_table(rs.sentence_text,
      '([()&!|]+') AS token,
```



```

        LATERAL regexp_matches(token, '^[a-zA-Z0-9]+)
        =([0-9]+)$') AS match
    WHERE token <> '' AND token <> '1'
),

variable_probabilities AS (
    -- Join with the probability table to obtain (var,
    val, prob)
    SELECT rv.var, rv.val, p.probability
    FROM relevant_variable_values rv
    JOIN triplet_probabilities_table p
    ON p.var = rv.var AND p.val = rv.val
),

joined_variable_probs AS (
    -- Combine sentence-variable pairs with their
    probabilities
    SELECT sv.sentence_id,
        CASE WHEN sv.var = '1' AND sv.val = 1 THEN 1.0
        ELSE vp.probability END AS prob
    FROM sentence_variables sv
    LEFT JOIN variable_probabilities vp
    ON sv.var = vp.var AND sv.val = vp.val
),

aggregated_sentence_probs AS (
    -- Compute the product of probabilities per sentence
    SELECT sentence_id, EXP(SUM(LN(prob))) AS
        sentence_probability
    FROM joined_variable_probs
    GROUP BY sentence_id
),

final_sentence_probabilities AS (
    -- Join sentence text with computed probabilities
    SELECT rs.sentence_text, ap.sentence_probability
    FROM ranked_sentences rs
    JOIN aggregated_sentence_probs ap
    ON rs.sentence_id = ap.sentence_id
)

-- Final output: original sentence ID, sentence text,
    computed probability
SELECT s.id, tostring(s._sentence) AS sentence_text, fp.
    sentence_probability
FROM sentences_table s
JOIN final_sentence_probabilities fp
    ON tostring(s._sentence) = fp.sentence_text
ORDER BY s.id;

```

B SYNTHETIC DATA EXAMPLE

A sample snippet of a table with 1000 rows and 100 different ID's. Every unique variable column is called L for some $L \in [a, b, \dots, z]$. Values in those columns are also randomized and take some value from L. After which their corresponding sentences are constructed and randomized probabilities are added to both DuBio, Tuple and Triplet storage. *Note: names are combinations of 2 names to increase amount of possible names for data generation*

id	name	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	_sentence
1	janebianca	r	i	k	g	r	o	s	j	x	c	e	p	w	a	z	q	k	f	c	Bdd((a1=0&b1=0&c1=0&d1=0&e1=0&f1=0&g1=0&h1=	
1	janebianca	g	t	i	n	m	d	c	z	w	e	y	d	q	o	f	h	n	y	s	e	Bdd((a1=1&b1=1&c1=1&d1=1&e1=1&f1=1&g1=1&h1=
1	janebianca	i	z	w	a	r	v	c	z	u	e	d	e	x	p	o	j	z	m	m	Bdd((a1=2&b1=2&c1=2&d1=2&e1=2&f1=2&g1=2&h1=	
1	janebianca	a	y	u	r	t	f	u	x	w	b	m	i	v	y	o	w	f	d	z	q	Bdd((a1=3&b1=3&c1=3&d1=3&e1=3&f1=3&g1=3&h1=
1	janebianca	e	y	l	t	n	b	y	m	v	b	j	r	t	w	t	x	i	x	i	Bdd((a1=4&b1=4&c1=4&d1=4&e1=4&f1=4&g1=4&h1=	
1	janebianca	q	p	h	e	y	r	v	r	i	t	o	s	b	h	k	q	c	j	g	Bdd((a1=5&b1=5&c1=5&d1=5&e1=5&f1=5&g1=5&h1=	
1	janebianca	x	d	k	q	m	d	g	o	o	d	m	g	v	r	p	z	h	w	p	Bdd((a1=6&b1=6&c1=6&d1=6&e1=6&f1=6&g1=6&h1=	
1	janebianca	p	d	g	q	m	o	b	q	f	x	i	p	t	z	f	a	b	j	s	k	Bdd((a1=7&b1=7&c1=7&d1=7&e1=7&f1=7&g1=7&h1=
1	janebianca	m	t	y	k	b	z	c	r	k	p	n	q	p	w	n	b	y	o	x	Bdd((a1=8&b1=8&c1=8&d1=8&e1=8&f1=8&g1=8&h1=	
1	janebianca	q	n	s	h	b	w	k	z	d	a	u	q	o	r	t	i	j	o	Bdd((a1=9&b1=9&c1=9&d1=9&e1=9&f1=9&g1=9&h1=		
2	phoebesheila	u	a	n	h	j	p	x	b	a	o	v	y	g	c	w	l	a	g	y	Bdd((a2=0&b2=0&c2=0&d2=0&e2=0&f2=0&g2=0&h2=	
2	phoebesheila	z	w	w	k	t	e	q	y	k	d	y	f	i	d	z	s	a	c	i	b	Bdd((a2=1&b2=1&c2=1&d2=1&e2=1&f2=1&g2=1&h2=
2	phoebesheila	b	r	u	h	e	x	o	t	i	a	f	o	x	h	o	a	r	n	r	h	Bdd((a2=2&b2=2&c2=2&d2=2&e2=2&f2=2&g2=2&h2=
2	phoebesheila	o	h	x	k	h	k	a	g	m	i	v	h	f	i	v	z	z	m	i	Bdd((a2=3&b2=3&c2=3&d2=3&e2=3&f2=3&g2=3&h2=	
2	phoebesheila	p	m	w	t	s	w	r	t	h	k	a	d	x	k	f	i	y	i	r	r	Bdd((a2=4&b2=4&c2=4&d2=4&e2=4&f2=4&g2=4&h2=
2	phoebesheila	h	j	c	c	g	a	o	w	s	a	c	e	k	y	r	i	x	q	b	n	Bdd((a2=5&b2=5&c2=5&d2=5&e2=5&f2=5&g2=5&h2=
2	phoebesheila	v	t	m	k	z	m	p	c	n	x	y	d	i	k	g	i	f	x	Bdd((a2=6&b2=6&c2=6&d2=6&e2=6&f2=6&g2=6&h2=		
2	phoebesheila	j	q	w	a	b	r	z	m	o	h	d	e	a	s	j	q	p	h	d	Bdd((a2=7&b2=7&c2=7&d2=7&e2=7&f2=7&g2=7&h2=	
2	phoebesheila	p	d	d	b	i	j	n	e	y	g	e	p	o	n	e	m	h	b	d	Bdd((a2=8&b2=8&c2=8&d2=8&e2=8&f2=8&g2=8&h2=	
2	phoebesheila	g	o	w	z	i	m	x	o	f	b	o	u	i	a	e	y	k	s	o	Bdd((a2=9&b2=9&c2=9&d2=9&e2=9&f2=9&g2=9&h2=	
3	stephaniecrystal	u	p	x	u	m	e	y	i	b	z	i	o	m	x	j	v	e	w	c	o	Bdd((a3=0&b3=0&c3=0&d3=0&e3=0&f3=0&g3=0&h3=
3	stephaniecrystal	v	c	j	v	w	e	r	q	k	w	s	u	e	y	k	j	i	y	Bdd((a3=1&b3=1&c3=1&d3=1&e3=1&f3=1&g3=1&h3=		
3	stephaniecrystal	v	k	c	h	t	p	b	c	i	h	k	o	e	w	c	t	n	v	b	d	Bdd((a3=2&b3=2&c3=2&d3=2&e3=2&f3=2&g3=2&h3=
3	stephaniecrystal	y	k	c	b	e	r	l	q	h	a	q	v	i	x	t	i	k	i	z	Bdd((a3=3&b3=3&c3=3&d3=3&e3=3&f3=3&g3=3&h3=	
3	stephaniecrystal	b	v	s	q	h	e	u	m	v	p	p	q	i	g	h	d	a	c	Bdd((a3=4&b3=4&c3=4&d3=4&e3=4&f3=4&g3=4&h3=		
3	stephaniecrystal	u	m	d	o	y	a	a	n	r	s	y	a	o	n	x	i	q	w	b	e	Bdd((a3=5&b3=5&c3=5&d3=5&e3=5&f3=5&g3=5&h3=
3	stephaniecrystal	s	k	c	a	m	f	u	i	v	z	a	y	m	a	i	x	q	p	q	Bdd((a3=6&b3=6&c3=6&d3=6&e3=6&f3=6&g3=6&h3=	
3	stephaniecrystal	a	h	m	y	q	i	s	q	e	i	j	f	d	c	r	u	i	b	Bdd((a3=7&b3=7&c3=7&d3=7&e3=7&f3=7&g3=7&h3=		
3	stephaniecrystal	c	p	x	i	a	k	d	n	f	i	s	q	s	c	k	g	a	n	k	Bdd((a3=8&b3=8&c3=8&d3=8&e3=8&f3=8&g3=8&h3=	

Fig. 8. A snippet of a table with 1000 rows and 100 different ID's.